RimJobWorld - Sexual Harassment — Quadruple-Pass Codebase Audit

Scope: 84 C# files / 19,152 LOC. Passes: (1) hot paths & tick polling, (2) IMGUI / UI / VRAM, (3) inter-mod compatibility depth + expansion surface, (4) frameworks, caching, duplication. **Constraint honoured throughout:** no proposed change alters the aesthetic or the function of the UI. **Nothing has been built.** This is analysis only.

---

0. Executive summary

Three structural conclusions, in order of impact:

A. The VRAM problem is not IMGUI — it is `PortraitsCache`. IMGUI does not own VRAM; it references textures other systems allocate. The mod's real GPU footprint is `RenderTexture` allocations made by `RimWorld.PortraitsCache`, and the Command deck feeds that cache *unquantized, resize-derived, and animation-derived* sizes. RimWorld pools expired portrait RenderTextures in `renderTexturesPool` and **never destroys them** outside `PortraitsCache.Clear()` (game load/quit). One sustained drag of the Harem window's resize grip can therefore strand on the order of **hundreds of MB of VRAM for the rest of the session**. This is the single highest-value finding in the audit.

B. The IMGUI problem is draw calls and CPU, not memory. `Window_Harem` issues an estimated **400–900** `GUI.DrawTexture` **calls per frame**, across ~20 distinct textures, with **zero scroll culling** and **no** `EventType.Repaint` **gating**, and it rebuilds its entire pet roster (a cross-map pawn scan + 5 list allocations + one `ToLowerInvariant()` per pawn) **on every IMGUI event** — which is 2–6× per rendered frame, not once. Every one of these is fixable with byte-identical output on screen.

C. Soft-dependency detection is silently failing for Steam-installed mods — right now, on this machine. `ModLister.GetActiveModWithIdentifier(id)` matches the *exact* packageId. Steam-subscribed mods carry `ModMetaData.SteamModPostfix = "_steam"`. Only the Karma check passes `ignorePostfix: true`. Cross-referenced against the live 909-mod active list, **6 of 11 soft-dep probes currently evaluate to** `false` **despite the mod being installed and active**: Facial Animation, SpeakUp, RimTalk, Simple Slavery Collars, BondageBed Torture, and Rimpsyche-Sexuality. The entire FA bridge, the chatter routing, and the SSC/BondageBed conditioning layers are dead code in this configuration. This is a ~15-line fix.

Headline verdict on the stated goal ("UI runs better than vanilla with less overhead"): achievable, and by a comfortable margin — but *only* after items A and B. In its current state the Command deck is meaningfully heavier than a vanilla tab of equivalent density, primarily because of unculled rows, unquantized portraits, and per-event roster rebuilds. None of the three requires an aesthetic change.

PASS 1 — Hot paths, ticks, and multi-polling

1.1 Convergent modulus spike at `% 2500`

`MapComponent_HarassmentScan.MapComponentTick` and `GameComponent_Harassment.GameComponentTick` both key several heavy passes to the **same** modulus. On every tick where `TicksGame % 2500 == 0`, all of the following run back-to-back in one frame:

System	Location
`ConditioningUpkeep()` — full humanlike scan, `ApplyConditioningHediff` + `IsolationConditioningTick` + `BondageBedTick` per pawn	MapComponent
`RecomputeHeadGirls(map)` — full spawned scan + 2 `Dictionary` + 1 `List` allocation	MapComponent
`ControlUpkeep` `breakoutTick` block — `ReconcileOwnerRelations`, `RaidTraumaTick`, `KarmaDriftTick`, `DepthStageTick`, `DepthTraumaTick`, `SyncAttributeHediffs`, `DepthRivalry/Pecking/Codependency/Training` per profiled pet	MapComponent

System	Location
`foreach (profiles.Values)` — hypnosis decay + `RecordHistorySample` + `sex.HourlyDrift` for **every profile ever created**	GameComponent

That is 4+ full passes converging on one tick, **per map**, producing a periodic frame hitch roughly every 41 seconds of game time. The individual passes are each defensible; the collision is not.

Fix: phase-offset each system by a distinct constant — `(now + 617) % 2500 == 0`, `(now + 1291) % 2500 == 0`, etc. Behaviour is unchanged (same cadence, different phase); the spike flattens into four small costs spread across 2500 ticks. Trivially safe.

Secondary collisions: `% 500` (`EvilKeyScavenge`) vs `% 550` (`PhotoScavenge`) coincide every 5500 ticks; `% 250` (`autoTick`) vs `% 500` coincide on every other `autoTick`.

1.2 Periodic scans bypass the per-tick index that already exists

`PawnIndex.cs` builds `_humanlikes` and `_profiled` snapshots at most once per tick, and `ControlUpkeep` already forces that build on most ticks. Five periodic passes nevertheless re-scan `map.mapPawns.AllPawnsSpawned` raw and re-derive the same filters:

Pass	Cadence	Line
`RecomputeHeadGirls`	2500	`HarassmentEngine.cs:829`
`HeadGirlTick`	1200	`HarassmentEngine.cs:857`
`MemoryReactionScan`	350	`HarassmentEngine.cs:5971`
`EvilKeyScavenge`	500	`HarassmentEngine.cs:4175, 4195`
`PhotoScavenge`	550	`HarassmentEngine.cs:4284`

On a map with 200 spawned animals and 20 humanlikes this iterates 220 entries where 20 (or fewer, for `_profiled`) would do — an order-of-magnitude waste, five times over, at four different cadences. The index is already paid for; these passes simply do not consume it.

Fix: expose `Humanlikes()` / `Profiled()` accessors on `MapComponent_HarassmentScan` (calling `EnsureIndex()` internally, exactly as `PawnById` does) and swap the five loop sources. No behaviour change — all five already skip non-humanlikes or non-profiled pawns explicitly in their first `continue`.

1.3 `MemoryReactionScan` is the most expensive recurring call in the mod

Every 350 ticks, for every profiled pawn holding a `harasserMemory` entry, it runs `GenSight.LineOfSight(v.Position, h.Position, map)` per memory entry. `LineOfSight` is a cell-by-cell Bresenham walk. The distance pre-check (`> 9f` → skip) is present and correct, which caps the damage, but with a colony of well-remembered tormentors the inner loop is unbounded by design.

Fix: cap work per pass (process N pawns per invocation, round-robin via a stored cursor) and hoist the `LineOfSight` call behind the existing `bestCount` comparison so it only runs for the *candidate* tormentor rather than every entry. Same outcome, a fraction of the raycasts.

1.4 Unbounded profile store

`GameComponent_Harassment.profiles` is a `Dictionary<int, PawnProfile>` with **no pruning path anywhere in the codebase** (`grep` for `profiles.Remove` / `PruneProfiles` returns nothing). A profile is created for any pawn that gets seeded, including every raider, visitor, and trader who is ever harassed and then leaves forever. Consequences:

- The dictionary grows monotonically for the life of the save.
- It is `Scribe_Collections.Look(..., LookMode.Deep)` — every dead profile, its `chronicle` (capped at 100

entries) and its history samples are **fully serialised into every save file**.

- `GameComponentTick`'s hourly loop calls `sex?.HourlyDrift(false)` for **every** profile unconditionally,

including thousands that will never be seen again.

Fix: a low-cadence sweep (e.g. every 6 in-game days) that drops any profile whose pawn resolves to `null` via `PawnLookup.AnyMap` **and** is absent from `Find.WorldPawns`, is not a nemesis, and has no live owner link. Must be conservative — dropping a world-pawn's profile would erase nemesis grudges — so the predicate matters more than the sweep.

1.5 Tick-layer scorecard

ID	Finding	Severity	Certainty
T1	Four heavy passes converge on `% 2500` → periodic hitch	High	**98%**
T2	Five periodic scans bypass the existing `_humanlikes` / `_profiled` index	High	**99%**
T3	`MemoryReactionScan` runs `LineOfSight` per memory entry, every 350t	Med-High	**92%**
T4	`RecomputeHeadGirls` allocates 2 `Dictionary` + 1 `List` per map per call	Medium	**99%**
T5	`profiles` dictionary never pruned → save bloat + growing hourly loop	Medium	**93%**
T6	`%500` / `%550` scavenger collision	Low	**95%**

PASS 2 — IMGUI, UI, and VRAM

2.1 The portrait RenderTexture leak (highest-value finding)

`PortraitsCache.Get` keys its cache on a `PortraitParams` struct comprising **size × cameraOffset × cameraZoom × rotation × flags**, then per-pawn inside that bucket. Each miss allocates `new RenderTexture(w, h, 24)` at `requestedSize × 1.25 (supersample) × Prefs.UIScale`. Entries expire after 1 second of non-use into `renderTexturesPool` — and **`renderTexturesPool` is only ever emptied by `PortraitsCache.Clear()`**, which runs on game load and quit. Nothing trims it during play.

Three separate call patterns in this mod feed that cache unbounded keys:

(a) Resize-derived stage doll size — the severe one. `Window_Harem.DrawStage` computes `frame` from `rect.width` and `dollArea.height`, both derived from the **resizeable** window rect, then passes `new Vector2(frame.width, frame.height)` straight to `PortraitsCache.Get`. Every distinct pixel size touched during a resize drag mints a new RenderTexture.

At a 1200×800 window the stage frame is ≈216 × 556 → ×1.25 → 270 × 695 ≈ 188k px. At ~8 bytes/px (ARGB32 colour + 24-bit depth) that is **≈1.5 MB per size, per pawn**. A single deliberate corner-drag at 60 fps easily touches 150–300 distinct sizes:

> **≈ 225–450 MB of orphaned VRAM per sustained resize drag, held until the game is restarted.**

(b) Animated `cameraZoom` — the spiky one. `_stageZoom` is `Mathf.Lerp`-ed toward the target every Repaint and passed as `cameraZoom`. The code already quantises with `Mathf.Round(zoom * 50f) / 50f`, which is good instinct, but 1/50 granularity across the 0.82 → 1.70 Stylist transition still yields **up to 44 distinct `PortraitParams` buckets**. The 1-second expiry is longer than the ~0.5 s animation, so all 44 are live simultaneously:

> **≈ 66 MB live spike every time the Stylist is opened**, then 44 stage-sized RTs retained in the pool.

(c) Eight distinct headshot sizes. `DrawPortrait` is called with rects of 30, 31, 34, 36, 42, 44, 56 and 64 px — **8 buckets**, each a separate RenderTexture family per pawn, and each requiring its own full `PawnCacheRenderer.RenderPawn` pass. ≈189 KB/pawn across all eight, versus ≈71 KB if collapsed to two buckets.

Fixes — all pixel-identical on screen:

1. **Quantise the stage frame** to a 16 px step before the `Get` call, then `GUI.DrawTexture` the result into

the exact (unquantised) rect. The RT is already 1.25× supersampled; bilinear downscale of ≤16 px is not perceptible. Cuts resize churn ~16×. Optionally quantise to 64 px *while* `Event.current.type` indicates an active drag, and settle to 16 px on mouse-up — effectively zero churn.

1. **Stop animating `cameraZoom`.** Request the portrait at the *target* zoom only, and animate the

destination `Rect` instead (scale the quad). For a 0.5 s transition this is visually equivalent and collapses 44 buckets to 2. If the camera-zoom feel must be preserved exactly, quantise to 1/8 instead of 1/50 (8 buckets, ~12 MB).

1. **Collapse the 8 headshot sizes to 2** (e.g. 32 and 64) and let `GUI.DrawTexture` scale into the real rect.

~2.4 MB saved on a 20-pet colony and, more importantly, six fewer full pawn re-renders per pawn.

2.2 No `EventType.Repaint` gating

Unity IMGUI invokes `OnGUI` at least twice per rendered frame (Layout + Repaint) plus once per input event. `Dialog_Stylist` and `Dialog_DressUp` correctly guard their portrait fetch with `if (Event.current.type == EventType.Repaint)`. **`Window_Harem.DrawPortrait` and `DrawStageDoll` do not.**

Because `PortraitsCache.SetAnimatedPortraitsDirty()` marks any flashing pawn dirty every frame, a damaged or recently-hit pet on the roster triggers a **full `PawnCacheRenderer.RenderPawn` on each of 2–6 passes per frame** instead of one. Every decorative draw in the window (border icons, stage corners, bars, dividers) likewise executes on all passes despite being invisible on non-Repaint events.

Fix: gate purely decorative draws and all `PortraitsCache.Get` calls behind a Repaint check. Interactive widgets (`Widgets.ButtonInvisible`, `TextField`, scroll views) must continue to run on every pass — their hit-testing depends on it — so this is a targeted gate, not a blanket wrap.

2.3 Zero scroll culling

`grep` for `GetScrollPositionForRect`, visible-rect overlap tests, or any culling idiom in `Window_Harem` returns **nothing**. Every scroll view draws every row:

- **Roster column** (`DrawRosterColumn`): all `shown` rows at ~48 px each, ~14 draw calls per row

(background, selection accent, 4-call border, portrait, star + its backing box, 2× `FillableBar` at 2 textures each, risk dot).

- **Harem table** (`DrawHarem`): rows are **200 px tall** (40 px base + 160 px schedule). Each row draws a

portrait, two fillable bars, a submission bar, and a full 24-cell schedule grid. With 20 pets and ~4 rows visible, **16 rows of that are drawn entirely off-screen, every pass, on every frame.**

Fix: the standard vanilla idiom — compute the visible rect from the scroll position and `if (!rowRect.Overlaps(visibleRect)) continue;`. Pixel-identical output. On a 20-pet colony this is roughly a **5× reduction** in the Harem view's per-frame draw work, and it also stops off-screen pets from being `PortraitsCache.Get`-ed at all — which compounds directly into finding 2.1.

2.4 Per-color textures break IMGUI batching

`Window_Harem.SolidBar(Color)` and `SexualityPanelDrawer._texCache` memoise `SolidColorMaterials.NewSolidColorTexture(c)`, which allocates a **2×2 `Texture2D` per colour**. The caching is correct and the VRAM is negligible (~16 bytes each). The cost is elsewhere: Unity IMGUI batches consecutive draws that share a texture, and **every distinct bar colour forces a texture switch**. `Widgets.FillableBar` issues 2–3 `GUI.DrawTexture` calls with *different* textures for fill and background, so each bar is a guaranteed batch break — and the roster draws two bars per row.

Meanwhile `Widgets.DrawBoxSolid` — used everywhere else in the window — already draws `BaseContent.WhiteTex` tinted by `GUI.color`.

Fix: a small `FillBar(Rect, pct, fillColor, bgColor)` helper that draws `BaseContent.WhiteTex` twice with `GUI.color` set, replacing every `Widgets.FillableBar(..., SolidBar(x), SolidBar(y), ...)` call site. This collapses ~14 distinct textures to the one shared white texture, so all bars batch with all boxes. Byte-identical pixels; the current code is already drawing flat solid quads.

2.5 Decorative border trail is ~94 draw calls per pass

`DrawBorderIcons` walks all four window edges at a 40 px step. At 1200×800 that is ≈29 + 29 + 18 + 18 = **≈94 `GUI.DrawTexture` calls per pass**, purely ornamental, ungated by Repaint — so 190–560 calls per frame.

Fix (aesthetic preserved exactly): Repaint-gate it (immediate ~3× cut), and optionally cache the computed icon rects in a `List<Rect>` keyed on the window size so the loop arithmetic is not redone per pass. If a further cut is wanted later, the whole chrome layer can be

pre-rendered once into a window-sized `RenderTexture` and blitted as one quad — but that *costs* ~3.8 MB of VRAM, so it is only worth doing after 2.1 has freed hundreds of MB. Recommended as optional, not first-wave.

2.6 Roster rebuilt on every IMGUI event

`DoWindowContents` calls `AllPets()` unconditionally, and `DrawRosterColumn` then calls `FilterSortPets(pets)`. Per IMGUI event (2–6× per frame):

- `AllPets()` → `BuildGroups()` → a **cross-map** `AllPawnsSpawned` **scan**, plus a `Dictionary`, a `List<Group>`,

and a `List<Pawn>` per group; then de-duplicates with `list.Contains(...)` — **O(n²)**.

- `FilterSortPets()` → `new List<Pawn>(pets)`, a `FindAll` with a closure, **one `ToLowerInvariant()` string

allocation per pawn per event** for the filter, a `Sort` with a delegate, then two more `List<Pawn>` (`pinned` / `rest`).

- The `_sortMode == 3` comparator calls `HarassmentEngine.FindKeyHolderFor(a)` inside the comparison —

O(n log n) key-holder resolutions per event.

- `DrawSummaryCard` and `DrawHaremSummary` each independently re-walk the full pet list computing the same

averages.

Fix: the frame-stamp pattern already used correctly by `EnsureFontH` (`Time.frameCount == _fhFrame`) and by `FindLockedVictimForKey`. Cache the resolved+filtered+sorted list, the summary aggregates, and the lower-cased filter string against `Time.frameCount`, into reusable `List<Pawn>` fields rather than fresh allocations. Also swap `list.Contains` for a `HashSet<int>` of thing IDs. This is roughly a **6× reduction** in the window's CPU and allocation cost at zero visual difference.

2.7 `Verse.Window.WindowOnGUI` is patched globally to draw one grip

`Patch_HaremResizeGrip` postfixes `Verse.Window.WindowOnGUI` — one of the hottest methods in the game, invoked for **every window on the stack, on every IMGUI event**. The guard (`if (!(__instance is Window_Harem)) return;`) is cheap, but the Harmony stub, its try/finally, and the delegate dispatch are paid by every window belonging to every other mod, forever, so that this mod can draw three dark lines on one window.

Fix: `Window.WindowOnGUI` is `public virtual`. Override it in `Window_Harem`, call `base.WindowOnGUI()`, then draw the grip. The global patch is deleted outright. Identical visuals, zero cost to any other mod. This is the cleanest single win in the audit.

2.8 Gizmos rebuilt from scratch every frame

`Patch_Pawn_GetGizmos` is a postfix on a method RimWorld calls **every frame for every selected pawn**. It invokes five builders (`BuildKeyHolderGizmos`, `BuildConditionedGizmo`, `BuildFightBackGizmo`, `BuildAutoResistGizmo`, `BuildOnaholeTimerGizmo`), and `BuildKeyHolderGizmos` allocates a fresh `List<Gizmo>` plus a stream of `Command_Action` / `Command_Toggle` / `Command_Target` objects — each with its own label string concatenation, closure, and (for `Command_Target`) a `new TargetingParameters { validator = lambda }`. With a multi-pawn selection this is dozens of allocations per frame.

The inner `FindLockedVictimForKey` is already frame-cached, which shows the pattern is understood — it simply was not applied one level up.

Fix: memoise the assembled gizmo list per `(pawn.thingIDNumber, Time.frameCount)`. Gizmos are consumed immediately after construction, so a one-frame cache is safe. See also 3.3 for a correctness bug in that existing cache.

2.9 On the explicit question: IMGUI alternatives with less VRAM and the same aesthetic

Assessed honestly, including the options that do **not** work:

Option	VRAM effect	Aesthetic risk	Verdict
Portrait size + zoom quantisation (2.1)	**−100s of MB**	None (supersampled bilinear)	**Do first**
Scroll-rect culling (2.3)	Large indirect (off-screen pets never enter the portrait cache)	None	**Do first**

Option	VRAM effect	Aesthetic risk	Verdict
Repaint-gating decorative draws (2.2, 2.5)	None (CPU/draw-call win)	None	**Do first**
`BaseContent.WhiteTex` + `GUI.color` for bars (2.4)	Negligible VRAM; removes batch breaks	None	**Do**
Frame-stamped roster cache (2.6)	None (CPU + GC win)	None	**Do**
Pre-rendered chrome `RenderTexture` (2.5, optional)	**+3.8 MB**, −94 draw calls/pass	None	Optional, only after 2.1
`Widgets.DrawAtlas` 9-slice instead of `DrawBoxSolid` + `DrawBox`	Neutral	Low	**No** — `DrawAtlas` issues 9 draws; our borders are 5. Not a win.
Unity UIToolkit / UI Elements	N/A	N/A	**Not viable.** RimWorld 1.6 (Unity 2019.4) exposes no UIToolkit runtime path that can interleave with `Verse.WindowStack`'s IMGUI ordering.
Retained-mode `Mesh` / `GL` batching	Would help draw calls	High — clip-rect and z-order interactions with `GUI.BeginGroup` are fragile	**No**
Compressing our own `Textures/UI/*.png`	Small, real	None	Worth a pass; ~35 UI textures loaded eagerly as `static readonly` at startup

The core correction to the premise: IMGUI is not the VRAM consumer here. Vanilla RimWorld's UI is IMGUI too, and it is cheap. What makes this window heavier than vanilla is (i) portrait RenderTextures keyed on unquantised sizes, (ii) unculled rows, and (iii) per-event roster rebuilds. Fix those three and the Command deck lands *below* an equivalent-density vanilla tab, because vanilla does not cache its roster per frame at all.

2.10 UI scorecard

ID	Finding	Severity	Certainty
U1	Resize-derived stage doll size → orphaned RenderTextures, never freed	**Critical**	**95%**
U2	Animated `cameraZoom` → up to 44 live stage-sized RTs (~66 MB spike)	High	**93%**
U3	Eight distinct headshot sizes → 8 RT families + 8 renders per pawn	Med-High	**97%**
U4	No `EventType.Repaint` gating → 2–6× redundant portrait renders + decorative draws	High	**90%**
U5	Zero scroll culling; Harem rows are 200 px tall and always fully drawn	**Critical**	**99%**
U6	Per-colour 2×2 textures break IMGUI batching on every bar	Medium	**96%**
U7	`DrawBorderIcons` ≈94 ungated decorative draws per pass	Medium	**97%**
U8	`AllPets()` / `FilterSortPets()` rebuilt per IMGUI event, 5 allocs + N string allocs	High	**98%**
U9	`AllPets()` de-duplicates with `List.Contains` → $O(n^2)$	Low-Med	**99%**
U10	`Verse.Window.WindowOnGUI` patched globally for one window's grip	Medium	**99%**
U11	`Patch_Pawn_GetGizmos` rebuilds all `Command_*` objects every frame	Med-High	**96%**

PASS 3 — Inter-mod compatibility

3.1 Steam `_steam` suffix breaks 6 of 11 soft-dep probes (confirmed live)

`ModLister.GetActiveModWithIdentifier(identifier, bool ignorePostfix = false)` looks up `modsByPackageId` — an **exact** match — unless `ignorePostfix: true` selects `modsByPackageIdIgnorePostfix`. `ModMetaData.SteamModPostfix` is `"_steam"`, and RimWorld marks the non-suffix-aware `ModsConfig` helpers `[Obsolete("Callers should use ... which automatically trims _steam suffixes")]`.

`SoftDeps.Detect()` passes `ignorePostfix: true` **only for Karma**. Cross-referencing against the live 909-mod active list:

Probe	Install source here	Result
`Nals.FacialAnimation`	Workshop	❌ **false** — entire FA bridge never initialises
`JPT.speakup`	Workshop	❌ **false**
`cj.rimtalk`	Workshop	❌ **false**
`TRIBeagle.simpleslaverycollars`	Workshop	❌ **false** — SSC collars unrecognised
`Mlie.BondageBedTorture`	Workshop	❌ **false** — `BondageBedTick` inert
`Maux36.Rimpsyche.Sexuality`	Workshop	❌ **false** — orientation gate falls back
`rim.job.world.onahole.ext`	Local	✅ true
`rjw.quirks`, `rjw.sexperience`, `Vegapnk.rjw.genes`	Local	✅ true
`astryl.KarmaReputation`	Workspace (+ `, true`)	✅ true

The mod's own startup log line would read `FacialAnim=False` on this machine despite Facial Animation being active at load order #31. **Fix: add `, true` to every probe** (~15 lines). Highest value-per-byte change in the entire audit.

3.2 Negative reflection results are not cached

The idiom `if (_type == null) _type = GenTypes.GetTypeInAnyAssembly("X");` treats "not found" as "not yet looked up". When the mod is **absent**, `_type` stays `null` forever and every call re-walks every loaded assembly. Live instances:

- `OnaholeBedTypeCached()` and `BeOnaholeJobType()` — both called from `ComputeIsInOnaholeBed`, which is one

of the hottest predicates in the mod (`PawnFlagCache` memoises it per tick, which mercifully caps this, but the first call each tick per pawn still pays a full assembly scan when Onahole Extension is not installed).

- `_coffeeCompType` (`HarassmentEngine.cs:6375`), `_rmbBest` / `_rmbSex` — the `_rmbBestTried` flag is the

correct pattern and is used for `_rmbBest`; the others lack it.

Fix: a `bool _tried` sentinel alongside each cached `Type`, matching the pattern already used correctly in `FABridge`, `KarmaBridge`, `RoomServiceBridge` and `_rmbBestTried`.

3.3 `FindLockedVictimForKey` roots a dead game graph

```
Dictionary<rjw.CompHoloCryptoStamped, int>  _lockedVictimFrame
Dictionary<rjw.CompHoloCryptoStamped, Pawn> _lockedVictimCache
```

Both are `static`, keyed on a **live** `ThingComp` **reference**, and store a `Pawn` **value**. Neither is cleared on game load. A `ThingComp` reaches its parent `Thing`, which reaches `Map`, which reaches `Game` — so after a save→load in the same session these dictionaries pin the entire previous game object graph. The 512-entry cap bounds the count, not the retained graph.

The Schematic explicitly records that `PawnFlagCache` keys on `int` ids "so the static store cannot root a dead Game graph". This cache predates or missed that rule.

Fix: key on `keyComp.parent.thingIDNumber` and store the victim's `thingIDNumber`, resolving through `PawnLookup` on read; clear both on `GameComponent.FinalizeInit`.

3.4 Compat surfaces that are already robust

Worth stating explicitly, because they are the template for the rest: `RoomServiceBridge` and `GastronomyBridge` (`VenueBridges.cs`), `FABridge`, `KarmaBridge`, `ReputationBridge`, `RimTalkBridge`, `SexHistoryBridge` and `FreeWillBridge` all use the correct shape — one-shot `_tried` init, `AccessTools` resolution cached into `MethodInfo`, every call wrapped in `try/catch`, graceful no-op when absent, and driving the **host mod's own validated job** rather than constructing one. `RoomServiceBridge` in particular is exemplary. The recommendation for pass 3 is to bring the remaining ad-hoc reflection sites up to this standard, not to redesign it.

3.5 Unexploited integration surface (all confirmed **active** in the live 909-mod list)

Ranked by fit-to-theme × (near-)zero overhead. All are additive, all fail closed if the mod is absent.

Mod	packageId	Opportunity	Overhead	Certainty
Vanilla Skills Expanded	`vanillaexpanded.skills`	The Phase E roadmap item — `ExpertiseDef` for Devoted endgame roles (Bodyguard/Handler/Courtesan). Confirmed installed; the planned hediff fallback is already the right design.	XML defs + one grant on stage transition — **zero tick cost**	96%
Suppression (Continued)	`Mlie.Suppression`	Directly overlaps `SlaveryHooks.SyncSuppression`. Both write a suppression floor. **Check for double-application before it becomes a bug report.**	Detection only	80%
Prisoners Dont Have Keys	`Mlie.PrisonersDontHaveKeys`	Manipulates prisoner inventories/keys — plausible interaction with the Holokey inventory logic in `BuildKeyHolderGizmos`. **Verify, do not assume.**	Detection only	70%
Nudity Matters More (+ opinions)	`dord.nuditymattersmore`, `shark510.nuditymattersmoreopinions`	`forceNudity` / `StripToBondage` should feed its nudity opinion model — makes forced nudity land socially instead of only as our own thought.	Event-driven	88%
Privacy, Please!	`abscon.privacy.please`	`FindPrivateCell` / `DragToPrivate` could defer to its room privacy evaluation instead of our own heuristic — better results, less code.	Replaces existing work	82%
RimHUD	`Jaxe.RimHUD`	Conditioning / rapport as custom HUD rows. RimHUD supports def-driven custom rows.	Def-only	85%
Grudges!	`TheOcean.Grudges`	`RegisterNemesis` could register a real grudge, so escaped pets carry hostility through the host mod's own system.	Event-driven	78%
Rimpsyche - Disposition	`Maux36.Rimpsyche.Disposition`	Active but **not probed** — only Rimpsyche core and Sexuality are. Disposition is exactly the axis `HarasserWillingness` wants.	Read-only, cached	84%
Restraints / Simple Restraint Belt	`BDew.Restraints`, `amegakull.SimpleRestraintBelt`	Recognise as locked harassment gear in `WearingLockedHarassmentGear`.	Def list, one-time	86%

Mod	packageId	Opportunity	Overhead	Certainty
Slaves Are Furniture / Pawns on Display	`Arkymn.SlavesAreFurniture`, `nawjak.pawnsondisplay`	Overlaps `boundInPublic` / display glances — either interop or explicitly stand down to avoid double-display.	Detection only	72%
Better Pawn Control	`VouLT.BetterPawnControl`	Harem schedule presets could ride its policy-switch event so pet schedules follow colony alerts.	Event-driven	70%
Ideology: More Precepts / Alpha Memes / VIE-Memes	`llunak.MorePrecepts`, `Sarg.AlphaMemes`, `VanillaExpanded.VMemesE`	`RJWSH_Ownership` meme could gain compatible precepts and appear in their generated ideologies.	XML only	80%
Epitaph / In Memoriam	`telardo.Epitaph`, `astryl.InMemoriam`	Owner-death legacy and the per-pawn chronicle could write real epitaphs/memorials — strong thematic payoff.	Event-driven	76%
Vanilla Social Interactions Expanded	`VanillaExpanded.VanillaSocialInteractionsExpanded`	Our `InteractionDef`s could register with its social log and relationship layer.	XML + detection	74%
Melee Animation	`co.uk.epicguru.meleeanimation`	Listed as a soft dep in the Schematic, but **no bridge code exists** — either wire it or drop the claim.	—	90%
RJW ecosystem, unprobed	`rjw.menstruation`, `vegapnk.cumpilation`, `c0ffee.rjw.events`, `rjw.dirtytalk`, `moth.rjw.flavor`, `rjw.sexperience.ideology`, `ElToro.*`	All active. Several have data our chronicle/attribute layer would benefit from; `rjw.dirtytalk` and `moth.rjw.flavor` overlap our banter and may double up.	Mixed	70%

3.6 Compat scorecard

ID	Finding	Severity	Certainty
C1	`_steam` suffix breaks 6 of 11 soft-dep probes — **live on this machine**	**Critical**	**97%**
C2	Negative `GetTypeInAnyAssembly` results re-scan all assemblies every call	Medium	**94%**
C3	`FindLockedVictimForKey` static cache roots a dead `Game` graph across load	Med-High	**90%**
C4	Melee Animation claimed as a soft dep with no implementing code	Low	**90%**
C5	Suppression / Prisoners-Dont-Have-Keys overlap unverified	Medium	**75%**

PASS 4 — Frameworks, caching, duplication

4.1 The UI widget layer is duplicated instead of shared

`ModernStyle` owns only `DrawCard`, the palette, and the scrollbar push/pop. Every actual widget lives as a `private static` inside `Window_Harem` and is therefore re-implemented elsewhere:

Widget	Canonical	Duplicate(s)
Gray button	`Window_Harem.GrayButton` (×2 overloads)	`Dialog_DressUp.GrayBtn`, `Dialog_Stylist.GrayBtn`
Section header	`Window_Harem.MiniHeader` (used 15×)	`SexualityPanelDrawer.Section`
Labelled bar	`Window_Harem.DrawMiniBar`	`SexualityPanelDrawer.Bar`, `SexualityPanelDrawer.SubDomBar`
Colour→texture cache	`Window_Harem._barCache`	`SexualityPanelDrawer._texCache`
Portrait draw	`Window_Harem.DrawPortrait`	`Dialog_DressUp`, `Dialog_Stylist` (each inline, each with different params)

This is a direct violation of the project's **Frameworks Over Duplicated Code** rule, and it is also what allowed the portrait-size sprawl in 2.1c: five call sites, five sets of parameters, no single place to enforce quantisation.

Fix: promote the widget set into `ModernStyle` (or a new `RJWSH.UI.Widgets`) — `GrayButton`, `GrayToggle`, `IconButton`, `IconToggle`, `MiniHeader`, `FillBar`, `Portrait`, `Card`. Every fix in Pass 2 (Repaint gating, white-tex bars, portrait quantisation) then applies **once** and is inherited by every panel. This single refactor is the multiplier that makes the rest of the UI work cheap.

4.2 No central tick scheduler

Nine periodic systems each hardcode their own modulus inline in `MapComponentTick`, which is what produced the `% 2500` convergence (1.1) and makes the cadences invisible as a set.

Fix: a tiny `TickScheduler` holding `(interval, phase, Action)` tuples, registered once, dispatched from one loop. Phases are then assignable centrally and collisions become structurally impossible. It also gives a single place to hang the kill-switch and the per-system `try/catch` the project's **Robust On Error** rule requires — several of the nine currently rely on the caller's guard.

4.3 `HarassmentEngine.cs` is a 6,577-line / 363 KB single file

Already partially split (`.Depth`, `.Nemesis`, `.Legacy`, `.Blackmail`, `.Rituals` partials), which is the right direction. The remaining core file still holds target selection, gizmo construction, jobs, scavenging, conditioning, banter, photos, market and venue logic. This is a maintainability finding, not a performance one, but it is why duplicated helpers (`FindPawnById` × 5 aliases, five `AllPawnsSpawned` scan idioms) keep reappearing — the file is too large to see itself.

Fix: continue the existing partial-class split — `.Gizmos`, `.Scavenge`, `.Banter`, `.Targeting`. Zero behavioural risk, purely file moves. Per the project rule, each move needs a timestamped backup to `D:\RimJobWorld - Sexual Harassment\<YYYYMMDD-HHMMSS>\` first.

4.4 Caching that is already right

Credit where due — these are correct and should be the model: `PawnIndex.EnsureIndex` (tick-stamped rebuild), `PawnLookup` (O(1) id resolution), `PawnFlagCache` (tick-scoped, **int-keyed**, self-healing, delegate-hoisted to avoid per-call allocation), `Window_Harem.EnsureFontH` (frame-stamped), `_lockedVictimFrame` (frame-stamped). The gaps identified in this audit are all *non-adoption* of these existing patterns, not absence of them.

4.5 Framework scorecard

ID	Finding	Severity	Certainty
F1	UI widgets duplicated across 4+ files; no shared widget layer	High	**99%**
F2	No central tick scheduler; nine inline moduli	Medium	**95%**
F3	`HarassmentEngine.cs` at 6,577 lines despite existing partial split	Medium	**100%**
F4	Five `FindPawnById` aliases (all now correctly delegate — cosmetic only)	Low	**90%**

---

5. Consolidated priority table

Rank	ID	Finding	Effort	Risk	Certainty
1	C1	Add `ignorePostfix: true` to all soft-dep probes	~15 lines	None	**97%**
2	U1	Quantise stage-doll portrait size	~10 lines	None	**95%**
3	U5	Scroll-rect culling in both list views	~20 lines	None	**99%**
4	U2	Stop animating `cameraZoom`; animate the destination rect	~15 lines	Low	**93%**
5	U10	Override `WindowOnGUI`; delete the global `Window` patch	~12 lines	None	**99%**
6	U4	Repaint-gate portraits + decorative draws	~30 lines	Low	**90%**
7	U8	Frame-stamp the roster / filter / summary caches	~40 lines	Low	**98%**
8	T1	Phase-offset the four `% 2500` systems	~8 lines	None	**98%**
9	T2	Route the five periodic scans through `_humanlikes` / `_profiled`	~25 lines	Low	**99%**
10	U3	Collapse 8 headshot sizes to 2	~10 lines	None	**97%**
11	U6	`FillBar` on `WhiteTex` + `GUI.color`	~25 lines	None	**96%**
12	F1	Promote widgets into a shared `ModernStyle` / `Widgets` layer	~150 lines	Low	**99%**
13	U11	Frame-cache the gizmo list	~20 lines	Low	**96%**
14	C3	Re-key `FindLockedVictimForKey` to int ids; clear on load	~15 lines	None	**90%**
15	C2	`_tried` sentinels on negative type lookups	~15 lines	None	**94%**
16	T3	Cap + reorder `MemoryReactionScan` `LineOfSight`	~20 lines	Low	**92%**
17	U7	Repaint-gate + cache border-icon rects	~15 lines	None	**97%**
18	F2	`TickScheduler`	~80 lines	Low	**95%**
19	T5	Conservative profile pruning sweep	~40 lines	**Med**	**93%**
20	T4	Static scratch collections in `RecomputeHeadGirls`	~10 lines	None	**99%**

---

6. Proposed paths forward

Four self-contained tranches. Each is independently shippable and independently testable.

Tranche A — "Free wins" (items 1, 5, 8, 15, 20)

Soft-dep suffix fix, `WindowOnGUI` override, `% 2500` phase offsets, `_tried` sentinels, scratch collections. **~60 lines, no behaviour change, no aesthetic change, no risk.** Tranche A alone revives six dead compat bridges and removes a global patch on the game's hottest UI method.

Tranche B — "VRAM and frame time" (items 2, 3, 4, 6, 10)

Portrait quantisation, scroll culling, zoom-animation change, Repaint gating, headshot size collapse. **~85 lines.** This is where the stated goal is actually met: it eliminates the resize leak, cuts the Harem view's draw work by roughly 5×, and removes 2–6× redundant pawn renders per frame. Requires careful testing of the Stylist transition and of the resize drag, but no visual change is expected — worth a side-by-side screenshot check at 1× and 2× UI scale.

Tranche C — "The UI framework" (items 7, 11, 12, 13, 17)

Shared widget layer, frame-stamped roster cache, `FillBar`, gizmo cache, border-icon cache. **~250 lines, mostly mechanical.** Do this *after* B so the Pass-2 fixes get folded into the shared widgets as they are written, rather than being applied twice. This is the tranche that makes the window measurably lighter than a vanilla tab.

Tranche D — "Tick hygiene and compat depth" (items 9, 14, 16, 18, 19 + Pass 3.5)

Index adoption, cache re-keying, `MemoryReactionScan` capping, `TickScheduler`, profile pruning, plus the integration work from 3.5. **Largest and highest-variance.** Recommend splitting: D1 = the mechanical tick items; D2 = profile pruning (needs a careful predicate and a save-compat test); D3 = new integrations, starting with the two *risk* items (Suppression double-application, Prisoners-Dont-Have-Keys) before any of the additive ones, and then Vanilla Skills Expanded expertise as the flagship feature.

Open questions before any build

1. **Resize quantisation granularity** — 16 px is my recommendation; 8 px is safer visually and still gives an

8× reduction. Preference?

1. **Stylist zoom** — animate the destination rect (best), or keep camera-zoom animation at 1/8 quantisation

(safer, still ~5× fewer RTs)? The first is visually near-identical but not bit-identical during the 0.5 s transition.

1. **Profile pruning** — acceptable to drop profiles for pawns that are gone from both the maps and

`Find.WorldPawns` and hold no nemesis/owner link? This is the only item in the audit that can lose data.

1. **Tranche C sequencing** — should the shared widget layer land before Tranche B (cleaner, fixes applied

once) at the cost of delaying the VRAM fix?

Confidence in this audit

Static analysis against the decompiled 1.6 source and the live 909-mod active list. Findings marked ≥95% are verified against engine source (`PortraitsCache`, `SolidColorMaterials`, `ModLister`, `ModMetaData`, `Widgets.FillableBar`, `Window.WindowOnGUI`). The VRAM figures in 2.1 are **derived estimates**, not measured — the mechanism is certain, the megabyte totals depend on `Prefs.UIScale`, RenderTexture format, and how far the user actually drags. Recommend confirming with a profiler capture before/after Tranche B rather than trusting my arithmetic.